LẬP TRÌNH JAVA NÂNG CAO

GV :TRƯƠNG BÁ VĨNH

NỘI DUNG

- Chương 1: Lập trình Applets
- Chương 2: Lập trình giao diện với Swing
- Chương 3: Các luồng vào/ra dữ liệu
- Chương 4: Lập trình Cơ sở dữ liệu với JDBC
- Chương 5: Luồng và Đa luồng
- Chương 6: Servlet

Điểm Đánh giá

- Điểm quá trình: 40%
  - ❖ Chuyên cần: 20%
  - ❖Kiểm tra giữa kỳ: 80%
- Điểm cuối kỳ: 60%
- Thi tự luận
- Thời gian: 60 phút
- Được sử dụng tài liệu

Chương I

Lập trình Applets

Nội dung

1.1 Giới thiệu Applets và Java Client-Side

1.2 Chu kỳ sống của Applets

1.3 Tạo một Applet

1.4 Truyền tham số cho Applets

1.5 Thực thi Applets

1.6 Quản lý sự kiện trên Applets

Bài tập

Applet là gì?

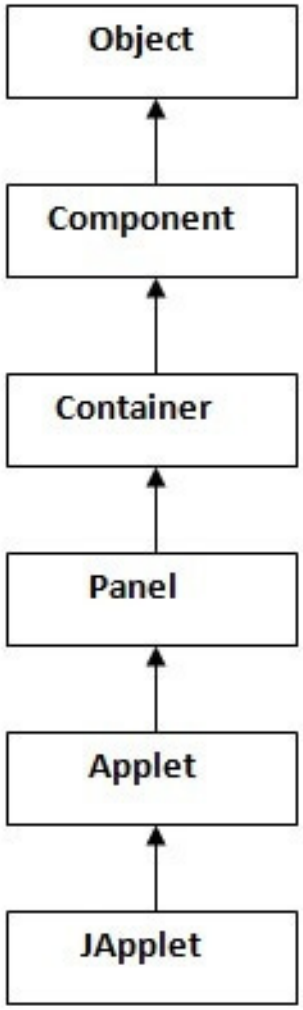
- Ngôn ngữ lập trình Java hiện nay được sử dụng khá phổ biến trên nhiều hệ điều hành khác nhau ở khắp nơi trên thế giới.
- Applet là những loại ứng dụng nhỏ được xây dựng bởi ngôn ngữ lập trình Java hoặc ngôn ngữ lập trình khác được biên dịch sang mã bytecode của ngôn ngữ Java.
- Applet có thể sử dụng trên nhiều hệ điều hành khác nhau như Microsoft Windows, Linux, MacOS, FreeBSD, Unix, v.v.

Lợi thế của Applet

- Có rất nhiều ưu điểm của Applet:
- Nó hoạt động ở phía máy khách nên thời gian phản hồi ít hơn.
- An toàn
- Nó có thể được thực thi bởi các trình duyệt chạy dưới nhiều dạng đĩa, bao gồm Linux, Windows, Mac Os, v.v.

Hạn chế của Applet

- Cần có plugin ở trình duyệt máy khách để thực thi applet.
- Hệ thống phân cấp của Applet



Applets

- Là một chương trình Java mà chạy với sự hỗ trợ của trình duyệt web
- Tất cả các applets là lớp con của lớp ‘Applet’
- Để tạo một applet, bạn cần import hai gói sau:
  - **java.applet**
  - **java.awt**

Vòng đời của Applet trong Java

Cấu trúc applet

- Định nghĩa một applet từ bốn sự kiện xảy ra trong quá trình thực thi
- Đối với mỗi sự kiện được định nghĩa bởi một phương thức tương ứng.
- Các phương thức:
  - **init( )**
  - **start( )**
  - **stop( )**
  - **destroy( )**
  - **paint()**

- Các phương thức khác:
  - **paint( )**
  - **repaint( )**
  - **showStatus( )**
  - **getAppletInfo( )**
- Các phương thức init(), start(), stop(), destroy(), and paint() được thừa kế từ applet.
- Mỗi phương thức này mặc định là rỗng. Vì thế các phương thức này phải được nạp chồng.

Làm thế nào để chạy một Applet?

- Có hai cách để chạy một applet:
- chạy Applet bằng file html.
- chạy Applet bằng tool appletViewer.

Biên dịch và thực thi applet

- Một applet thì được biên dịch theo cú pháp sau

  javac Applet1.java

- Để thực thi một applet, tạo một tập tin HTML có sử dụng thẻ applet
  - Thẻ applet có hai thuộc tính:
    - Width
    - Height
  - Để truyền tham số tới applet, sử dụng thẻ ‘param’, và tiếp theo là thẻ ‘value’
- Applet có thể được thực thi bằng applet viewer

Ví dụ

- Để thực thi Applet bằng file html, hãy tạo một Applet và biên dịch nó. Sau đó, tạo một file html và đặt mã Applet vào file html.

First.java

```
import java.applet.Applet;
import java.awt.Graphics;
public class First extends Applet{

public void paint(Graphics g){
g.drawString("welcome",150,150);
}

}
```

myapplet.html

```
<html>
<body>
<applet code="First.class" width="300" height="300">
</applet>
</body>
</html>
```

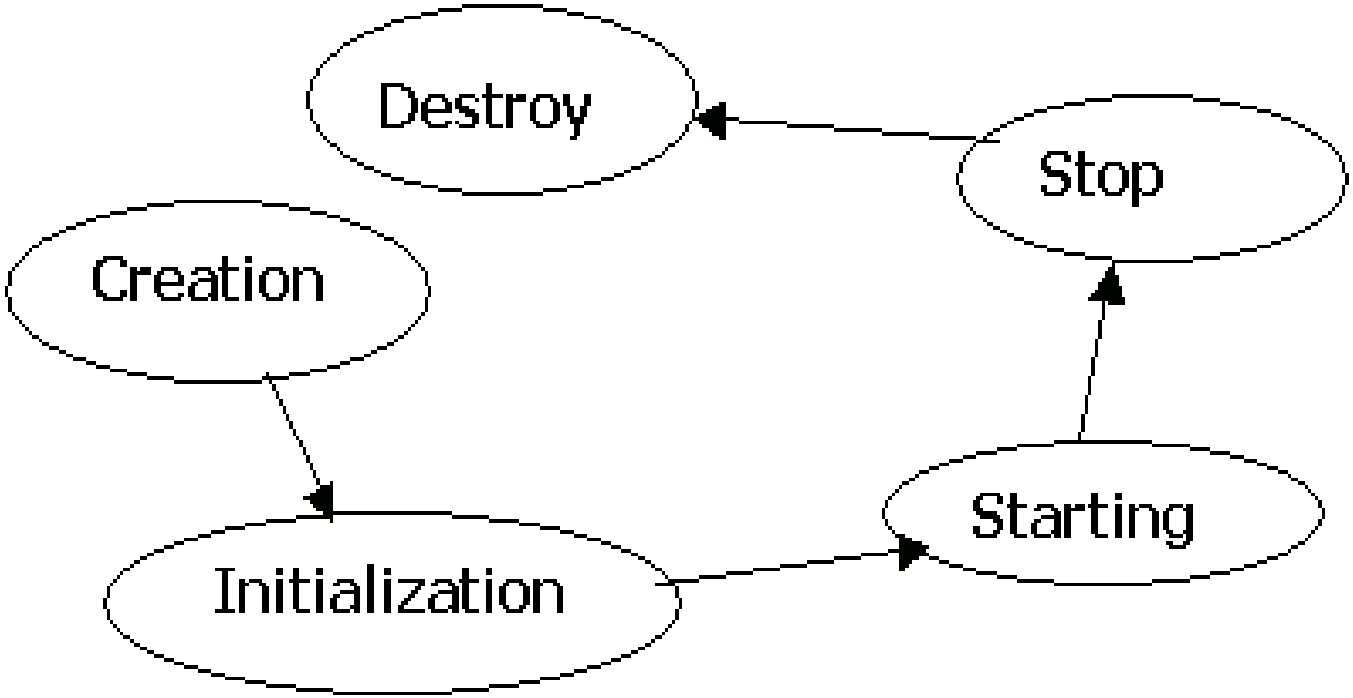
Điểm khác biệt giữa applet và một ứng dụng

- Các ứng dụng khi thực thi phải sử dụng trình biên dịch Java, trong khi các applets thực thi được trên bất kỳ trình duyệt nào mà hổ trợ Java, hoặc sử dụng ‘AppletViewer’ trong JDK.
- Một ứng dụng bắt đầu với phương thức ‘main()’. Còn đối với applet thì không sử dụng phương thức này
- Một ứng dụng sử dụng ‘System.out.println()’ để hiển thị, trong khi môt applet thì sử dụng phương thức ‘drawstring()’ để hiển thị.

Những hạn chế về bảo mật trong applet

- Không thể đọc hoặc viết các tập tin trên hệ thống tập tin của người sử dụng
- Không thể giao tiếp với một site trên internet. Mà chỉ giao tiếp với một dịch vụ trên trang web có applet.
- Không thể chạy bất kỳ chương trình nào trên hệ thống của người đọc
- Không thể load bất kỳ chương trình nào được lưu trên hệ thống của người sử dụng

Chu trình sống của applet



Life cycle of an applet

Truyền tham số tới một applet

- Để truyền tham số, sử dụng PARAM trong thẻ HTML
- Ví dụ

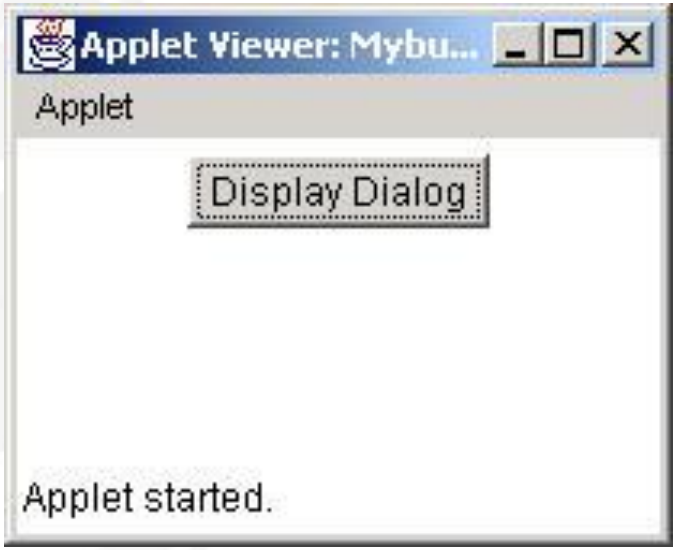
```
<applet code = "Mybutton1" width = “100” height = “100”>
<PARAM NAME = “Mybutton” value = “Display Dialog”>
</applet>
```

Ví dụ Truyền tham số cho Applet

Mybutton1.java

```
import java.awt.*;
import java.applet.*;
public class Mybutton1 extends Applet
{
        Button bNext;
        public void init()
        {
                String str = getParameter("mybutton");
                if (str==null)
                        str = new String ("Default");
                  bNext = new Button(str);
                add (bNext);
        }
}
```

```
<applet code = "Mybutton1" width = “100” height = “100”>
<PARAM NAME = “Mybutton” value = “Display Dialog”>
</applet>
```



Lớp đồ họa

- Được cung cấp bởi gói AWT
- Cung cấp một tập hợp các phương thức để vẽ như sau:
  - Oval
  - Rectangle
  - Square
  - Circle
  - Lines
  - Text in different fonts

Graphical Background

- Các phương thức để vẽ nền :
  - **getGraphics( )**
  - **repaint( )**
  - **update(Graphics g)**
  - **paint(Graphics g)**

Hiển thị chuỗi, ký tự và bytes

- Phương thức để vẽ hoặc hiển thị một chuỗi trên frame

  Cú pháp

  - **drawString(String str, int xCoor, int yCoor);**
- Phương thức để vẽ hoặc hiển thị các ký tự trên frame

  Cú pháp

  - **drawChars(char array[ ], int offset, int length, int xCoor, int yCoor);**
- Phương thức để vẽ hoặc hiển thị bytes trên frame

  Cú pháp

  - **drawBytes(byte array[ ], int offset, int length, int xCoor, int yCoor);**

```
import java.awt.*;
class FontMetricsUse extends Frame {
        public FontMetricsUse()     {
                super ("Detail of Fonts");
                setSize (400, 300);
                setVisible(true);
        }
        public void paint (Graphics g)   {
                Font f1 = new Font ("Times Roman", Font.PLAIN, 22);
                FontMetrics fm = g.getFontMetrics (f1);
                String name = fm.getFont().getName();
                g.drawString ("Details of Font " + name, 30, 50);
                g.drawString ("Height: " + String.valueOf (fm.getHeight()), 50, 75);
                g.drawString ("Ascent: " + String.valueOf (fm.getAscent()), 50, 100);
                g.drawString ("Descent: " + String.valueOf (fm.getDescent()), 50, 125);
                g.drawString ("Leading: " + String.valueOf (fm.getLeading()), 50, 150);
                Font f2 = new Font ("DialogInput", Font.PLAIN, 22);
                fm = g.getFontMetrics (f2);
                name = fm.getFont().getName();
                g.drawString ("Details of Font " + name, 30, 175);
                g.drawString ("Height: " + String.valueOf (fm.getHeight()), 50, 200);
                g.drawString ("Ascent: " + String.valueOf (fm.getAscent()), 50, 225);
                g.drawString ("Descent: " + String.valueOf (fm.getDescent()), 50, 250);
                g.drawString ("Leading: " + String.valueOf (fm.getLeading()), 50, 275);
        }
        public static void main (String args[])   {
                new FontMetricsUse ();
        }
}
```

Vẽ các hình thể

- Phương thức được sử dụng để vẽ đường thẳng như sau

  Cú pháp

  - **drawLine(int x1, int y1, int x2, int y2);**

- Các phương thức được sử dụng để vẽ đường tròn như sau

  Cú pháp

  - **drawOval(int xCoor, int yCoor, int width, int height);**
  - **setColor(Color c);**
  - **fillOval(int xCoor, int yCoor, int width, int height);**

- Phương thức sử dụng để vẽ hình vuông:

  Cú pháp

  - **drawRect(int xCoor, int yCoor, int width, int height);**
  - **fillRect(int xCoor, int yCoor, int width, int height);**

- Các phương thức được sử dụng để vẽ hình vuông có góc tròn

  Cú pháp

  - **drawRoundRect(int xCoor, int yCoor, int width, int height, int arcWidth, int arcHeight);**
  - **fillRoundRect (int xCoor, int yCoor, int width, int height, int arcWidth, int arcHeight);**

3D Rectangles & Arcs

- Các phương thức được sử dụng để vẽ hình 3D **Cú pháp**
  - **draw3DRect(int xCoord, int yCoord, int width, int height, boolean raised);**
  - **drawArc(int xCoord, int yCoord, int width, int height, int arcwidth, int archeight);**
  - **fillArc(int xCoord, int yCoord, int width, int height, int arcwidth, int archeight);**

Drawing PolyLines

- Các phương thức được sử dụng để vẽ nhiều được thẳng

Cú pháp

- **drawPolyline(int xArray[ ], int yArray[ ], int totalPoints);**
- **g.setFont(new Font("Times Roman", Font.BOLD,15));**

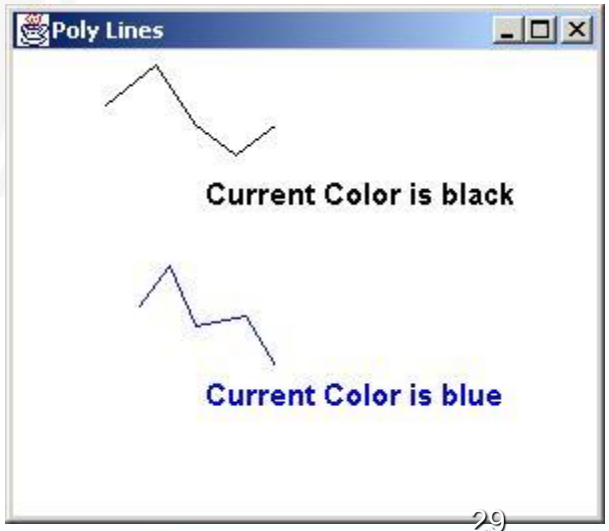
vẽ polyline.

```
import java.awt.*;

class PolyLines extends Frame
{
        int x1[] = {50, 75, 95, 115, 135};
        int y1[] = {50, 30, 60, 75, 60};
        int x2[] = {67, 82, 95, 120, 135};
        int y2[] = {150, 130, 160, 155, 180};

        public PolyLines()//constructor
        {
                super ("Poly Lines");
                setSize (300, 300);
                setVisible (true);
        }

        public void paint (Graphics g)
        {
                g.drawPolyline (x1, y1, 5);
                g.setFont (new Font("Times Roman", Font.BOLD, 15));
                g.drawString("Current Color is black", 100, 100);
                g.setColor(Color.blue);
                g.drawPolyline (x2, y2, 5);
                g.drawString ("Current Color is blue", 100, 200);
        }
        public static void main (String args[])
        {
                new PolyLines();
        }
}
```



Vẽ và tô các hình đa giác

- Các phương thức để vẽ và tô các hình đa giác

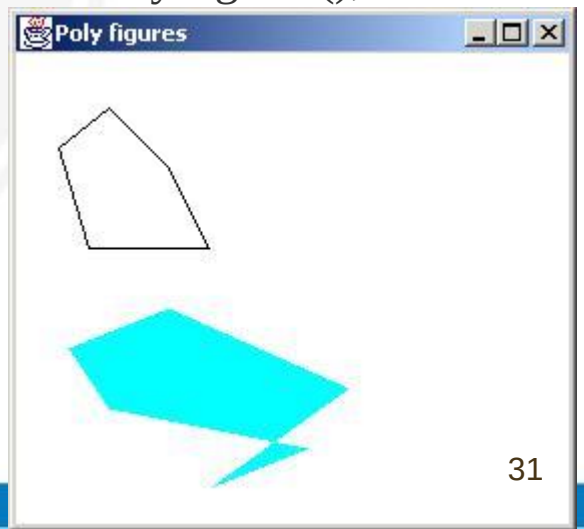
Cú pháp

- **drawPolygon(int x[ ], int y[ ], int numPoints);**
- **fillPolygon(int x[ ], int y[ ], int numPoints);**

Vẽ đa giác

- Chương trình dưới đây lấy các điểm từ 2 mảng để vẽ đa giác. Phương thức ‘drawPolygon()’ nhận 3 tham số sau giống như phương thức drawPolyline()

```
import java.awt.*;
class PolyFigures extends Frame
{
        int x1[] = {50, 25, 40, 100, 80};
        int x2[] = {80, 30, 50, 150, 100, 170};
        int y1[] = {50, 70, 120, 120, 80};
        int y2[] = {150, 170, 200, 220, 240,190};
        public PolyFigures()
        {
                super ("Poly figures");
                setSize(300, 300);
                setVisible (true);
        }
public void paint (Graphics g)
        {
                g.drawPolygon (x1, y1, 5);
                g.setColor (Color.cyan);
                g.fillPolygon (x2, y2, 6);
        }
        public static void main (String args[])
        {
                new PolyFigures();
        }
}
```



Màu

- Java sử dụng màu RGB
- Bảng các giá trị màu

Element	Range
Red	0-255
Green	0-255
Blue	0-255

- Cú pháp của hàm dựng để tạo một màu

color(int red, int green, int blue);

- Bảng trình bày các giá trị màu RGB thông thường

Color	Red	Green	Blue
White	255	255	255
Light Gray	192	192	192
Gray	128	128	128
Dark Gray	64	64	64
Black	0	0	0
Pink	255	175	175
Orange	255	200	0
Yellow	255	255	0
Magenta	255	0	255

Font

- Gói java.awt package cung cấp bởi lớp 'Font'
- Các phương thức của lớp Font:
  - **getAllFont( )**
  - **getLocalGraphicsEnvironment( )**
  - **getFont( )**
  - **getFontList( )**

- Hàm dựng Font nhận 3 tham số
  - Tên font trong chuỗi định dạng; tên này có trong phương thức getFontList( ).
  - Kiểu của font. Ví dụ như: Font.BOLD, Font.PLAIN, Font.ITALIC
  - Kích thước của font.
- Ví dụ

```
Font f1 = new Font("SansSerif", Font.ITALIC, 16);
g.setFont(f1);
```

Lớp FontMetrics

- Đo lường các ký tự khác nhau hiển thị trong các font khác nhau.
- Việc đo lường bao gồm ‘height’, ‘baseline’, ‘ascent’, ‘descent’ và ‘leading’ của font.
- Nó không cụ thể vì nó là một lớp trừu tượng

Lớp FontMetrics (tiếp theo…)

- Phương thức:
  - **getFontMetrics(f1)**
  - **getHeight( )**
  - **getAscent( )**
  - **getDescent( )**
  - **getLeading( )**
  - **getName( )**

Kiểu vẽ

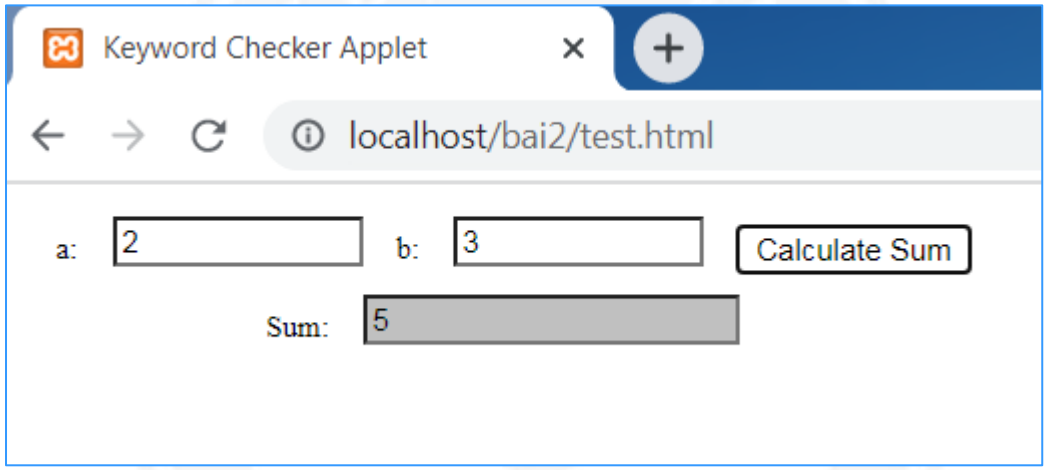
- Các đối tượng để vẽ được sử dụng.
- Method used to make old and new contents visible on the screen

  setXORMode(Color c)
- Method used to revert to the overwrite mode

  setPaintMode( )

Bài tập 1

- Viết applet như sau:Nhập vào 2 số a và b. Tính tổng 2 số

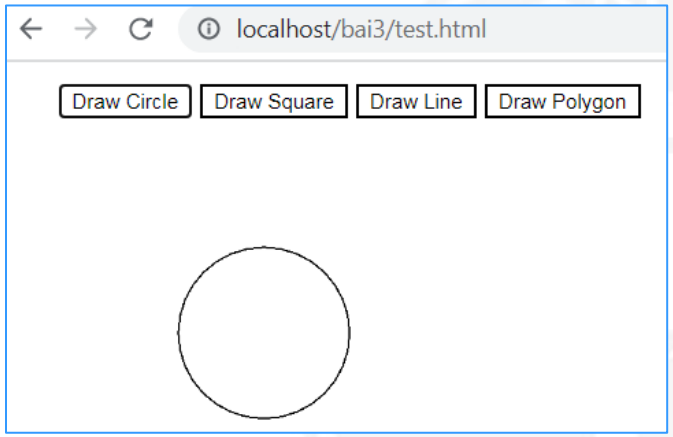


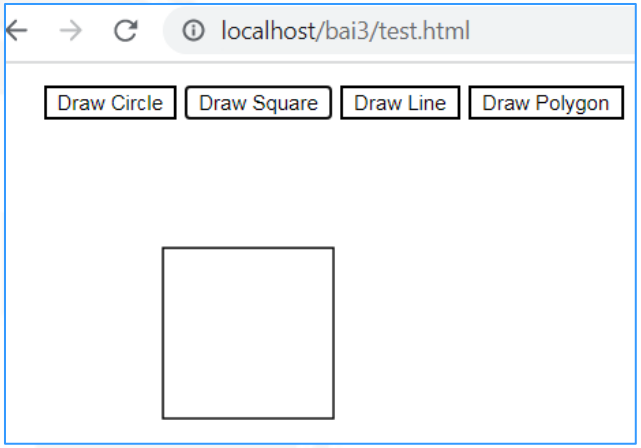
```java
import java.applet.Applet;
import java.awt.*;
import java.awt.event.*;
public class SumApplet extends Applet implements ActionListener {
    TextField aField, bField, resultField;
    Button sumButton;
    public void init() {
        setLayout(new FlowLayout());
        aField = new TextField(columns:10);
        bField = new TextField(columns:10);
        resultField = new TextField(columns:15);
        resultField.setEditable(b:false);
        sumButton = new Button(label:"Calculate Sum");
        add(new Label(text:"a:"));
        add(aField);
        add(new Label(text:"b:"));
        add(bField);
        add(sumButton);
        add(new Label(text:"Sum:"));
        add(resultField);
        sumButton.addActionListener(this);
    }
    public void actionPerformed(ActionEvent e) {
        try {
            // Read the numbers from the text fields
            int a = Integer.parseInt(aField.getText());
            int b = Integer.parseInt(bField.getText());
            int sum = a + b;
            resultField.setText(String.valueOf(sum));
        } catch (NumberFormatException ex) {
            resultField.setText(t:"Invalid input");
        }
    }
}
```

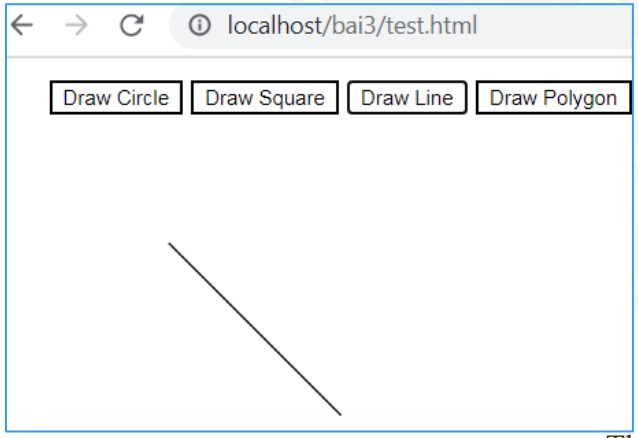
```html
<html>
<head>
    <title>Tính tổng hai số</title>
</head>
<body>
    <applet code="SumApplet.class" width="400" height="150">
    </applet>
</body>
</html>
```

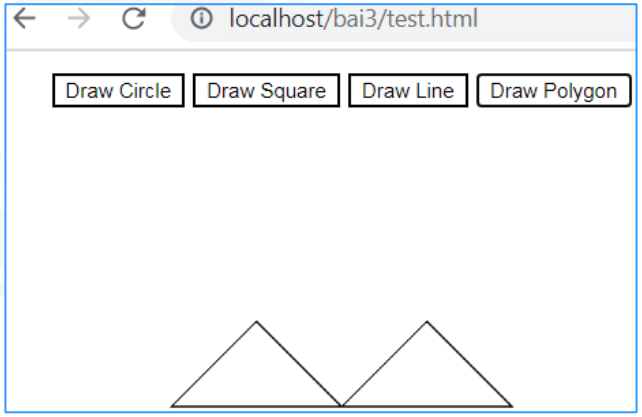
Bài tập 2

- Viết applet như sau: Tạo các nút hình tròn, hình vuông, đường thẳng, đa giác. Khi chọn nút nào thì vẽ hình đó.









```java
import java.applet.Applet;
import java.awt.*;
import java.awt.event.*;
public class ShapeDrawerApplet extends Applet implements ActionListener {
    private String shapeToDraw = "";
    Button circleButton, squareButton, lineButton, polygonButton;
    public void init() {
        // Set layout for the applet
        setLayout(new FlowLayout());
        // Create buttons
        circleButton = new Button(label:"Draw Circle");
        squareButton = new Button(label:"Draw Square");
        lineButton = new Button(label:"Draw Line");
        polygonButton = new Button(label:"Draw Polygon");
        // Add buttons to applet
        add(circleButton);
        add(squareButton);
        add(lineButton);
        add(polygonButton);
        // Add action listeners to the buttons
        circleButton.addActionListener(this);
        squareButton.addActionListener(this);
        lineButton.addActionListener(this);
        polygonButton.addActionListener(this);
    }
```

```java
    public void actionPerformed(ActionEvent e) {
        if (e.getSource() == circleButton) {
            shapeToDraw = "circle";
        } else if (e.getSource() == squareButton) {
            shapeToDraw = "square";
        } else if (e.getSource() == lineButton) {
            shapeToDraw = "line";
        } else if (e.getSource() == polygonButton) {
            shapeToDraw = "polygon";
        }
        repaint();
    }
    public void paint(Graphics g) {
        g.setColor(Color.BLACK);
        switch (shapeToDraw) {
            case "circle":
                g.drawOval(x:100, y:100, width:100, height:100);
                break;
            case "square":
                g.drawRect(x:100, y:100, width:100, height:100);
                break;
            case "line":
                g.drawLine(x1:100, y1:100, x2:200, y2:200);
                break;
            case "polygon":
                int[] xPoints = {100, 150, 200, 250, 300};
                int[] yPoints = {200, 150, 200, 150, 200};
                g.drawPolygon(xPoints, yPoints, xPoints.length);
                break;
```

```html
<html>
<head>
    <meta charset="utf-8">
    <title>Vẽ hình cơ bản</title>
</head>
<body>
    <applet code="ShapeDrawerApplet.class" width="400" height="400">
    </applet>
</body>
</html>
```

Q/A ?